

Depth360: Monocular Depth Estimation using Learnable Axisymmetric Camera Model for Spherical Camera Image

Noriaki Hirose¹ and Kosuke Tahara¹

Abstract—Self-supervised monocular depth estimation has been widely investigated to estimate depth images and relative poses from RGB images. This framework is attractive for researchers because the depth and pose networks can be trained from just time sequence images without the need for the ground truth depth and poses.

In this work, we estimate the depth around a robot (360° view) using time sequence spherical camera images, from a camera whose parameters are unknown. We propose a learnable axisymmetric camera model which accepts distorted spherical camera images with two fisheye camera images. In addition, we trained our models with a photo-realistic simulator to generate ground truth depth images to provide supervision. Moreover, we introduced loss functions to provide floor constraints to reduce artifacts that can result from reflective floor surfaces. We demonstrate the efficacy of our method using the spherical camera images from the GO Stanford dataset and pinhole camera images from the KITTI dataset to compare our method's performance with that of baseline method in learning the camera parameters.

I. INTRODUCTION

Accurately estimating three-dimensional (3D) information of objects and structures in the environment is essential for autonomous navigation and manipulation [1], [2]. Self-supervised monocular depth estimation without ground truth (GT) depth images is one of the most popular approaches for obtaining 3D information [3]–[7]. In self-supervised monocular depth estimation, there are several limitations including that this method requires the camera parameters, there is no scaling of the estimated depth image, and it functions poorly for highly reflective objects.

This paper proposes a novel self-supervised monocular depth estimation approach for a spherical camera image view. We propose a learnable axisymmetric camera model to handle images from a camera whose camera parameter is unknown. Because our learnable camera model can handle unknown distorted images, such as spherical camera images based on two fisheye images, we can obtain 360° 3D point clouds all around a robot from only one camera.

In addition to self-supervised learning with real images, we rendered many pairs of spherical RGB images and their corresponding depth images from a photo-realistic robot simulator. In training, we mixed these rendered images with real images to achieve sim2real transfer in an attempt to provide scaling for the estimated depth. Moreover, we introduce additional cost functions to improve the accuracy of depth estimation for reflective floor areas. We provide supervision

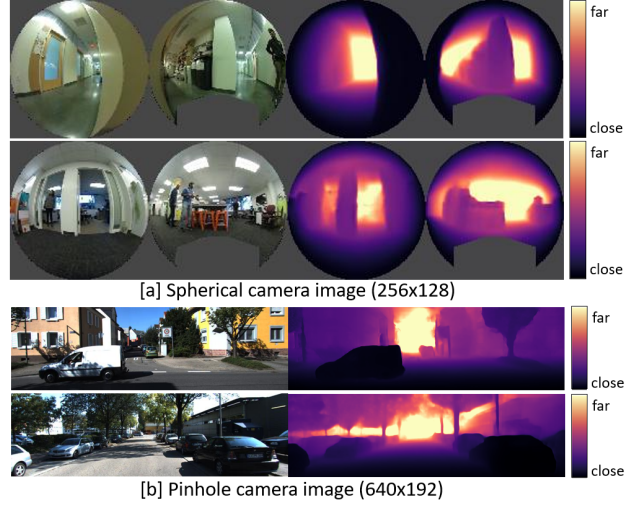


Fig. 1: Estimated depth images from 256×128 spherical camera images from the GO Stanford dataset and 640×128 pinhole camera images from the KITTI dataset. A spherical image was constructed using two fisheye images. The gray area in the spherical image is a common mask that does not consider these pixels for training. Our method can estimate depth from the spherical image in [a] and the pinhole camera image in [b] using our learnable axisymmetric camera model.

for estimated depth images from the future and past robot footprints, which are obtained from the reference velocities in the data collection. Main contributions are summarized as:

- A novel learnable axisymmetric camera model capable of handling distorted images with unknown camera parameters,
- Sim2real transfer using ground truth depth from a photo-realistic simulator to sharpen the estimated depth image and provide scaling,
- Proposal of novel loss functions that use the robot footprints and trajectories to provide constraints against reflective floor surfaces.

In addition, we blended front-and back-side fisheye images to reduce the occluded area for image prediction in self-supervised learning. As a result of the above approaches, our method can estimate the depth image without the large artifacts that often result from a low-resolution spherical image, thus allowing our method fast computation useful for online settings.

Our method was trained and evaluated on the GO Stanford (GS) dataset [9]–[11], as depicted in in Fig 1[a], with time sequence spherical camera images and associated reference velocities for dataset collection. Moreover, we evaluated our proposed axisymmetric camera model with the

¹Noriaki Hirose and Kosuke Tahara are in Toyota Central R&D Labs., Inc., 41-1, Yokomichi, Nagakute, Aichi, 480-1192, Japan hirose@mosk.tytlabs.co.jp

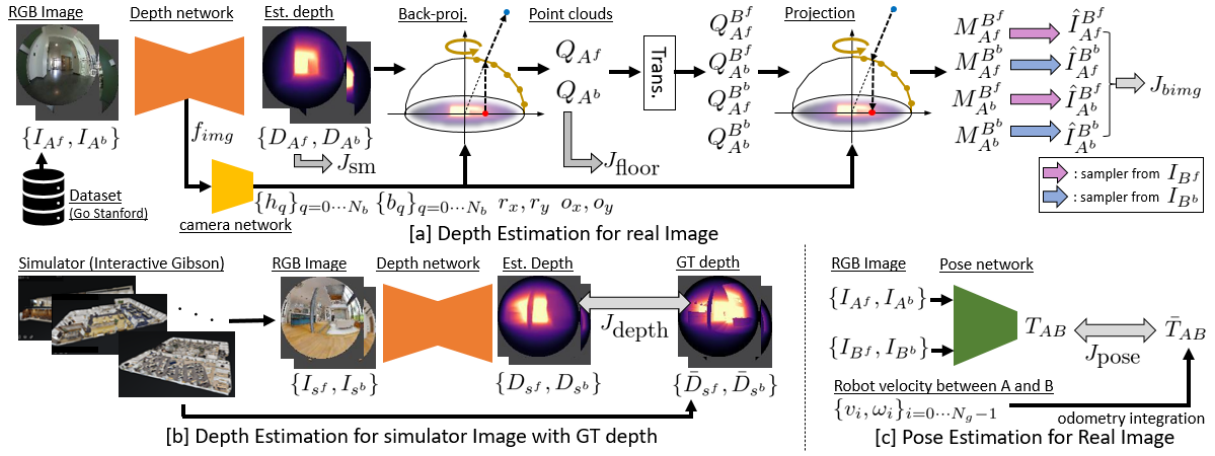


Fig. 2: **Block diagram of our method.** [a] Depth estimation for the real image of the GO Stanford dataset, [b] depth estimation for the simulation image with GT depth. The RGB image and GT depth are rendered via a 3D reconstructed environment using a Matterport scanner [8], [c] pose estimation for the “Trans.” in [a].

KITTI dataset [12], as depicted in Fig 1[b], to provide a comparison with other learnable camera models [13], [14]. We quantitatively and qualitatively evaluated our method with the GS and KITTI datasets.

II. RELATED WORK

Monocular depth estimation using self-supervised learning has been widely investigated by several approaches that use deep learning [4], [6], [13], [15]–[34]. Zhou et al. [3], and Vijayanarasimhan et al. [35] applied spatial transformer modules [36] based on the knowledge of structure from motion to achieve self-supervised learning from time sequence images. Since the publication of [3] and [35], several subsequent studies have attempted to estimate accurate depth using different methods, including a modified loss function [37], an additional loss function to penalize geometric inconsistencies [5], [7], [13], [38], [39], a probabilistic approach to estimate the reliability of depth estimation [34], [40], masking dynamic objects [4], [13], and an entirely novel model structure [19], [41], [42]. We review the two categories most related to our method.

Fisheye camera image. Various approaches have attempted to estimate depth images from fisheye images. [43] proposed supervised learning with sparse GT from LiDAR. [44]–[46] leveraged multiple fisheye cameras to estimate a 360° depth image. Similarly, Kumar et al. proposed self-supervised learning approaches with a calibrated fisheye camera model [47] and semantic segmentation [48].

Learning camera model. Gordon et al. [13] proposed a self-supervised monocular depth estimation method that can learn the camera’s intrinsic parameters. Vasiljevic et al. [14] proposed a learnable neural ray surface (NRS) camera model for use in highly distorted images.

In contrast to most related work [14], our learnable axisymmetric camera model can be fully differentiable without using the softmax approximation. Hence, end-to-end learning can be achieved without adjusting the hyperparameters during training. In addition, we provide supervision for the estimated depth images by using a photo-realistic simulator and robot trajectories from the dataset. As a result, our

method can accurately estimate depth from low-resolution spherical images.

III. PROPOSED METHOD

From the process in Fig. 2, we designed the following cost functions to train the depth and pose networks:

$$J = J_{bimg} + \lambda_d J_{depth} + \lambda_f J_{floor} + \lambda_p J_{pose} + \lambda_s J_{sm}. \quad (1)$$

Since our spherical camera can capture the 360° view around the robot, the occluded area between two consecutive frames is less than that when using a pinhole camera. To leverage this advantage, we propose J_{bimg} , which is an occlusion-aware image loss, inspired by monodepth2 [37]. J_{depth} penalizes the depth difference using the GT depth from photo-realistic simulator. By penalizing J_{depth} with J_{bimg} , our model can learn sim2real transfer and can thereby estimate accurate depths from real images. J_{floor} is the proposed loss function that provides supervision against floor areas by using robot’s footprint and trajectory. In addition, J_{pose} penalizes the difference between the estimated pose and the GT pose calculated from the reference velocities in the dataset. J_{sm} penalizes the discontinuity of the estimated depth using the exact same objective, following [16], [37].

In the following sections, we first present J_{bimg} and J_{pose} in the overview of our training process. Then, we explain our camera model, with J_{depth} and J_{floor} as our main novel contributions. We define the robot and camera coordinates based on the robot pose X , as shown in Fig. 3. Σ_{X^r} is the base coordinate of the robot. In addition, Σ_{X^f} and Σ_{X^b} are the coordinates of the front-and back-side fisheye cameras in the spherical camera, respectively. Our method assumes that the relative poses between each coordinate are known.

A. Overview

1) *Process of depth estimation:* Fig. 2[a] presents the calculation of depth estimation for real images. Because there are no GT depth images, we employed self-supervised learning approach using time sequence images. Unlike previous approaches [3], [37], our spherical camera can capture both the front-and back-side of the robot. Hence, we

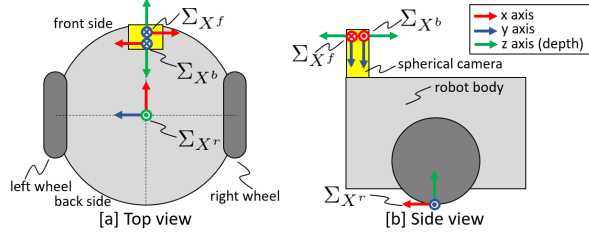


Fig. 3: **Robot and spherical camera coordinates.** Σ_{X^f} and Σ_{X^b} are the coordinates of the front-and back-side fisheye cameras in the spherical camera on the robot, respectively. Σ_{X^r} denotes the robot coordinate at pose X .

propose a cost function to blend the front-and back-side images and thereby reduce the negative effects of occlusion. We feed the front-side images I_{A^f} and back-side image I_{A^b} at camera pose A into the depth network $f_{\text{depth}}()$ to estimate the corresponding depth images as $D_{A^f}, D_{A^b} = f_{\text{depth}}(I_{A^f}, I_{A^b})$. Here, D_{A^f} and D_{A^b} are estimated for each camera coordinate Σ_{A^f} and Σ_{A^b} , respectively. By back-projection $f_{\text{backproj}}()$ with our proposed camera model, we can obtain the corresponding point clouds Q_{A^f} and Q_{A^b} .

$$Q_{A^f} = f_{\text{backproj}}(D_{A^f}), \quad Q_{A^b} = f_{\text{backproj}}(D_{A^b}) \quad (2)$$

“Trans.” in Fig. 1[a] transforms the coordinates of the estimated point clouds as follows:

$$\begin{aligned} Q_{A^f}^{B^f} &= T_{AB} Q_{A^f}, & Q_{A^f}^{B^b} &= T_{AB} \cdot T_{fb}^{-1} Q_{A^b}, \\ Q_{A^f}^{B^f} &= T_{fb} \cdot T_{AB} Q_{A^f}, & Q_{A^b}^{B^b} &= T_{fb} \cdot T_{AB} \cdot T_{fb}^{-1} Q_{A^b}. \end{aligned} \quad (3)$$

Here, $Q_{X^\alpha}^{Y^\beta}$ denotes the point clouds on the coordinate Σ_{Y^β} estimated from image I_{X^α} . T_{AB} is the estimated transformation matrix between Σ_{A^f} and Σ_{B^f} . T_{fb} is the known transformation matrix between Σ_{X^f} and Σ_{X^b} . By projecting these point clouds onto the image coordinates of I_{A^f} or I_{A^b} , we can estimate four matrices $M_{A^\alpha}^{B^\beta}$,

$$M_{A^\alpha}^{B^\beta} = f_{\text{proj}}(Q_{A^\alpha}^{B^\beta}) \quad (4)$$

where, $\alpha = \{f, b\}$ and $\beta = \{f, b\}$. According to [36], we estimate I_{A^α} by sampling the pixel value of I_{B^β} as $\hat{I}_{A^\alpha}^{B^\beta} = f_{\text{sample}}(M_{A^\alpha}^{B^\beta}, I_{B^\beta})$. Here $\hat{I}_{A^\alpha}^{B^\beta}$ denotes the estimated image of I_{A^α} by sampling I_{B^β} . Note that we estimated four images from the combination of $\alpha = \{f, b\}$ and $\beta = \{f, b\}$, as shown in Fig. 2[a]. We calculate the blended image loss J_{bimg} to penalize the model during training.

$$\begin{aligned} J_{\text{bimg}} &= \lambda_1 J_{L1} + \lambda_2 J_{SSIM}. \\ J_{L1} &= \sum_{\alpha \in \{f, b\}} f_{\min}(|I_{A^\alpha} - \hat{I}_{A^\alpha}^{B^f}|, |I_{A^\alpha} - \hat{I}_{A^\alpha}^{B^b}|), \\ J_{SSIM} &= \sum_{\alpha \in \{f, b\}} f_{\min}(Md_{\text{ssim}}(I_{A^\alpha}, \hat{I}_{A^\alpha}^{B^f}), Md_{\text{ssim}}(I_{A^\alpha}, \hat{I}_{A^\alpha}^{B^b})). \end{aligned} \quad (5)$$

Here, $f_{\min}(\cdot, \cdot)$ selects a smaller value at each pixel and calculates the mean of all the pixels. $d_{\text{ssim}}(\cdot, \cdot)$ is the pixel-wise structural similarity (SSIM) [49], following [37]. By selecting a smaller value in $f_{\min}(\cdot, \cdot)$, we can equivalently select the non-occluded pixel value of $\hat{I}_{A^\alpha}^{B^f}$ or $\hat{I}_{A^\alpha}^{B^b}$ to calculate L1 and SSIM [37]. In addition, M is a mask to remove

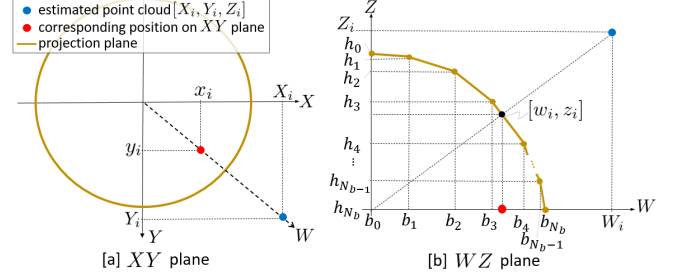


Fig. 4: **Projection plane of our learnable axisymmetric camera model.** Our projection plane using multiple line segments is axisymmetric and convex upward to being fully differentiable.

the pixels without RGB values and those of the robot itself, which are depicted in gray color in Fig. 1[a].

2) *Process of pose estimation:* Fig. 2[c] denotes the process to estimate T_{AB} . Unlike previous monocular depth estimation approaches, we use the GT transformation matrix $\bar{T}_{AB}$ from the integral of the reference velocities $\{v_i, \omega_i\}_{i=0 \dots N_g-1}$ to move between poses A and B . J_{pose} is designed as $J_{\text{pose}} = \sum (\bar{T}_{AB} - T_{AB})^2$.

B. Axisymmetric camera model

To handle the dataset without camera parameters and videos in the wild for depth estimation, we propose a novel camera model. Our proposed camera model, depicted in Fig. 4, has some constraints, in that the projection plane (yellow lines in Fig. 4) is axisymmetric and convex upwards. Following our design, it is a fully differentiable model without approximations by a softmax function.

In Fig. 4, $[X_i, Y_i, Z_i]$ is the i -th estimated 3D point, and $[x_i, y_i]$ is the corresponding position between -1.0 and 1.0 on the XY plane. We define $[x_i, y_i] = [\tilde{x}_i/r_x + o_x, \tilde{y}_i/r_y + o_y]$, where $[\tilde{x}_i, \tilde{y}_i]$ is the position on the image plane, r_x and r_y are learnable variables for the angle of view, and o_x and o_y are learnable variables for the offset between the image plane and the XY plane. The W axis is defined as the direction from the origin to $[X_i, Y_i, 0]$.

According to the above constraints, the projection plane on the XY plane can be depicted as a unit circle whose center is the origin, as shown in Fig. 4[a]. In addition, the projection plane on the WZ plane monotonically decreases. In our camera model, we designed the projection plane on the WZ plane with consecutive line segments by the points $[b_i, h_i]_{i=0 \dots N_b}$, as shown in Fig. 4[b].

1) *convex upward projection plane:* To derive $[b_i, h_i]_{i=0 \dots N_b}$ which ensures the convex upward projection plane, our camera parameter network $f_{\text{cam}}()$ estimates the camera parameters as follows:

$$\{\Delta b_i, \Delta h_i / \Delta b_i\}_{i=1 \dots N_b}, r_x, r_y, o_x, o_y = f_{\text{cam}}(f_{\text{img}}) \quad (6)$$

where $\Delta h_i = \tilde{h}_{i-1} - \tilde{h}_i$, $\Delta b_i = \tilde{b}_{i-1} - \tilde{b}_i$, and f_{img} are the image features from the depth encoder. $[b_i, \tilde{h}_i]$ are the i -th points before normalization.

In $f_{\text{cam}}()$, we provide a sigmoid function to achieve $\Delta h_i / \Delta b_i > 0$ and $\Delta b_i > 0$, to ensure a convex upward constraint. By simple algebra with $\tilde{h}_{N_b} = 0.0$ and $\tilde{b}_{N_b} = 1.0$, we can obtain $[\tilde{b}_i, \tilde{h}_i]_{i=0 \dots N_b}$. By performing

normalization to stabilize the training process, we can obtain $h_i = \tilde{h}_i / \sum_{k=0}^{N_b} \tilde{h}_k$ and $b_i = \tilde{b}_i / \sum_{k=0}^{N_b} \tilde{b}_k$.

2) *Back-projection*: Back-projection is the process of calculating the point clouds $[X_i, Y_i, Z_i]$ from the estimated depth Z_i at $[x_i, y_i]$. First, we calculate $[w_i, z_i]$, which is the intersection point on the projection plane. The line of the j -th line segment can be expressed as:

$$Z = \frac{\Delta h_j}{\Delta b_j} \cdot W + \frac{h_{j-1}b_j - h_j b_{j-1}}{\Delta b_j} = \alpha_j W + \beta_j. \quad (7)$$

To be a fully differentiable process, we calculate the intersection points between all lines of the projection plane and the vertical line $W = w_i$ ($= \sqrt{x_i^2 + y_i^2}$). Then, we select the minimum height at all intersections as z_i because the projection plane is upwardly convex.

$$z_i = \min(\{\alpha_j w_i + \beta_j\}_{j=1 \dots N_b}) \quad (8)$$

Note that the min function is a differentiable function. As a result, we can obtain $X_i = \frac{Z_i}{z_i} \cdot x_i$ and $Y_i = \frac{Z_i}{z_i} \cdot y_i$.

3) *Projection*: The projection process calculates $[x_i, y_i]$ from $[X_i, Y_i, Z_i]$. Similar to back-projection, we first calculate the intersection point $[w_i, z_i]$ and derive $[x_i, y_i]$. The line between the origin and $[X_i, Y_i, Z_i]$ can be expressed as $Z = \frac{Z_i}{W_i} \cdot W = \gamma_i \cdot W$. Much like the back-projection process, the minimum value at all intersections on the Z axis can be z_i . Hence, z_i and w_i can be derived as follows:

$$z_i = \min \left(\left\{ \frac{\gamma_i \beta_j}{\gamma_i - \alpha_j} \right\}_{j=1 \dots N_b} \right), \quad w_i = z_i / \gamma_i. \quad (9)$$

Thus, $[x_i, y_i] = [\frac{w_i}{W_i} \cdot X_i, \frac{w_i}{W_i} \cdot Y_i]$. Here, $W_i = \sqrt{X_i^2 + Y_i^2}$.

C. Floor loss

The highly reflective floor surface in indoor environments can often cause significant artifacts in depth estimation. Our floor loss function provides geometric supervision for floor areas. Assuming that the camera is horizontally mounted on the robot and its height is known, the floor loss function is constructed by two components: $J_{floor} = J_{fcl} + J_{lbl}$.

1) *Footprint consistency loss J_{fcl}* : The robot footprint is almost horizontally flat, and its height is obtained from the camera mounting position. According to the method shown below, we obtain the GT depth $\{\bar{D}_{A^\alpha}\}_{\alpha=\{f,b\}}$ only around the robot footprint between $\pm M_r$ steps and provide the supervision as follows:

$$J_{fcl} = \sum_{\alpha \in \{f,b\}} \frac{1}{N_m} \sum_{i=1}^{N_m} M_f |\bar{D}_{A^\alpha} - D_{A^\alpha}|, \quad (10)$$

where M_f is the mask, which masks out the pixels without the GT values in $\bar{D}_{A^\alpha}$, and N_m is the number of pixel with the GT depth. To make $\bar{D}_{A^\alpha}$, we take the point clouds of the robot footprint between $\pm M_r$ steps as follows:

$$Q_{foot} = (Q_{foot}^{-M_r}, \dots, Q_{foot}^0, \dots, Q_{foot}^{M_r}). \quad (11)$$

where $Q_{foot}^i = T^i Q_{foot}^0$ is the point cloud of the robot footprint at the pose of the i -th step, where Q_{foot}^0 is that at the origin (= pose A) and T^i is the transformation matrix

between the origin and i -th robot pose from the teleoperator's velocities, similar to T_{AB} . $\bar{D}_{A^\alpha}$ can be derived as:

$$\bar{D}_{A^\alpha}(x_j^\alpha, y_j^\alpha) = Z_j^\alpha, \quad (12)$$

where $[x_j^\alpha, y_j^\alpha] = f_{proj}(T_{r\alpha} Q_{foot}[j])$ and Z_j^α are the values of the Z axis of $T_{r\alpha} Q_{foot}[j]$. $T_{r\alpha}$ is the known transformation matrix from Σ_{X^r} to Σ_{X^α} . $Q_{foot}[j]$ is the j -th 3D point in Q_{foot} . By assigning all point clouds into $\bar{D}_{A^\alpha}$ using (12), we can take $\bar{D}_{A^\alpha}$ to calculate J_{fcl} . Note that $\bar{D}_{A^\alpha}$ is a sparse matrix. No GT pixel in $\bar{D}_{A^\alpha}$ is excluded by M_f in J_{fcl} .

2) *lower boundary loss J_{lbl}* : In indoor scenes, floor areas often reflect ceiling lights. This can cause it to appear that there are holes on the floor in the estimated depth image. To provide a lower boundary for the height of estimated point clouds, we observe two key points: 1) the camera is horizontally mounted on the robot, and 2) most objects around the robot are higher than the floor. One exception would be anything that is downstairs, which would be lower than the floor. However, it is rare in to find such occurrences in the dataset, because teleoperation around the stairs is dangerous and ill-advised.

To provide the constraint for the Y axis (= height) value of the estimated point clouds, J_{lbl} can be given as follows:

$$J_{lbl} = \sum_{\alpha \in \{f,b\}} \frac{1}{N} \sum_{i=1}^N (\max(0.0, Y_{A^\alpha} - h_{cam})), \quad (13)$$

where $Q_{A^\alpha} = [X_{A^\alpha}, Y_{A^\alpha}, Z_{A^\alpha}]$. J_{lbl} penalizes Y_{A^α} , which is larger (= lower) than the floor height (= h_{cam}).

D. Sim2real transfer with J_{depth}

In conjunction with self-supervised learning using real-time sequence real images, we used the GT depth image $\{\bar{D}_{sf}, \bar{D}_{sb}\}$ from a photo-realistic robot simulator [50], as shown in Fig. 2[b]. Although there is an appearance gap between real and simulated images, the GT depth from the simulator can help the model understand the 3D geometry of the environment from the image. Here, $J_{depth} = \sum_{\alpha \in \{f,b\}} d_{depth}(\bar{D}_{s^\alpha}, D_{s^\alpha})$, where $D_{sf}, D_{sb} = f_{depth}(I_{sf}, I_{sb})$. I_{sf} and I_{sb} are front-and back-side fisheye images, respectively, from the simulator. We employed the same metric $d_{depth}()$ as the baseline method [51] to measure the depth differences. To achieve a sim2real transfer, we simultaneously penalized J_{depth} and J_{bimg} .

IV. EXPERIMENT

A. Dataset

Our method was mainly evaluated using the GO Stanford (GS) dataset with time-sequence spherical camera images. In addition, we used the KITTI dataset with pinhole camera images for comparison with the baseline methods, which attempts to learn the camera parameters.

1) *GO Stanford dataset*: We used the GS dataset [9]–[11] with time sequence spherical camera images (256×128) and the reference velocities, which were collected by teleoperating turtlebot2 with Ricoh THETA S. The GS dataset contains 10.3 hours of data from twelve buildings at the Stanford

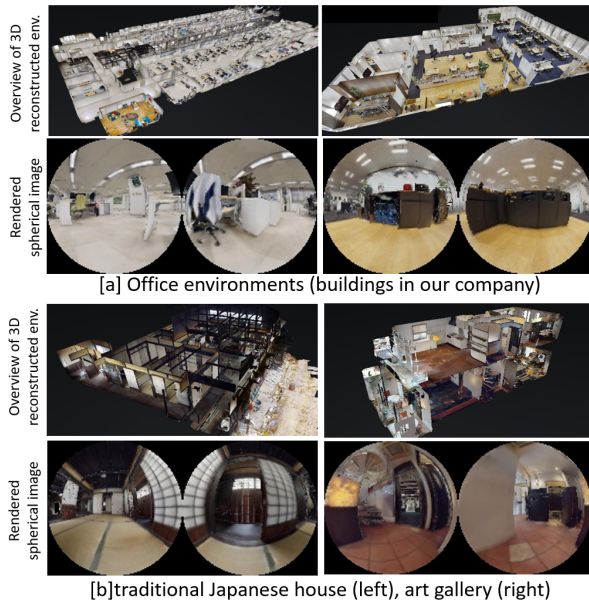


Fig. 5: Examples of measured 3D reconstructed environments [8] and rendered spherical images [50]. [a] The office and laboratory from our company’s buildings for training and validation, [b] traditional Japanese house and art gallery for testing.

University campus. To train our networks, we used a training dataset from eight buildings, following [11].

In addition to the real images of the GS dataset, we collected pairs of simulator images and GT depth images for J_{depth} . We measured 12 floors (e.g., office rooms, meeting rooms, laboratories) in our company buildings and ten environments (e.g., a traditional Japanese house, art gallery, fitness gym) for the simulator, as shown in Fig. 5. We separated them into groups: eight floors in our company buildings for training, four floors in our company buildings for validation, and ten environments not in our company for testing. For training, we rendered 10,000 data from each environment using interactive Gibson [50]. In addition, we collected 1,000 data points for testing from ten environments.

Data collection of the GT depth from a real spherical image is a challenging task. Hence, in the quantitative analysis, we used the GT depth from the simulator. To evaluate the generalization performance, our test environment was not derived from our company building. Examples of the domain gaps between training and testing are shown in Fig. 5. In the qualitative analysis, we used both real and simulated images.

2) *KITTI dataset*: To evaluate our proposed camera model against the baseline methods, we employed the KITTI raw dataset [12]. Similar to the baseline methods, we separated the KITTI raw dataset via Eigen split [53] with 40,000 images for training, 4,000 images for validation, and 697 images for testing. To compare with baseline methods, we employed the widely used 640×192 image size as input.

B. Training

In training with the GS dataset, we randomly selected 12 real images from the training dataset as $\{I_{A^\alpha}\}_{\alpha \in \{f,b\}}$. Then, we randomly selected $\{I_{B^\alpha}\}_{\alpha \in \{f,b\}}$ between $\pm 5(=$

TABLE I: Evaluation of depth estimation from the GO Stanford dataset. For the three leftmost metrics, smaller values are better; for the rightmost metric, higher values are better. “SGT” denotes the GT depth from the simulator. The bold values indicate the best results. All methods except $\dagger$ were evaluated without median scaling because scaling is learned from the SGT. $\ddagger$ used the half-sphere camera model shown in (14) and (15).

Method	SGT	Abs-Rel	Sq-Rel	RMSE	$\delta < 1.25$
monodepth2 [37] $\ddagger$		0.586	1.890	1.123	0.439
–with SGT [37], [51] $\ddagger$	✓	0.228	0.162	0.535	0.664
Alhashim et al. [51]	✓	0.203	0.154	0.542	0.698
Our method (full)	✓	0.198	0.143	0.525	0.711
–wo J_{depth} $\dagger$		0.377	0.335	0.829	0.433
–wo our cam. model $\ddagger$	✓	0.248	0.179	0.548	0.646
–wo J_{tbl}	✓	0.233	0.161	0.532	0.665
–wo J_{fel}	✓	0.216	0.151	0.530	0.684
–wo blending in J_{bimg}	✓	0.205	0.147	0.530	0.702
–wo J_{pose}	✓	0.200	0.145	0.532	0.698

TABLE II: Evaluation of depth estimation on the KITTI dataset. For the leftmost three metrics, smaller values are better; for the rightmost metric, higher values are better. The bold values indicate the best results. All the methods in this table employ a pretrained ResNet-18 for their encoder. All values were calculated after median scaling. $\dagger$ was trained on a mixed dataset with the KITTI, Cityscape, bike, and GO Stanford datasets. The others were trained only on KITTI.

Method	Camera	Abs-Rel	Sq-Rel	RMSE	$\delta < 1.25$
Gordon et al. [13]	known	0.129	0.982	5.23	0.840
Gordon et al. [13]	learned	0.128	0.959	5.23	0.845
NRS [52]	known	0.137	0.987	5.337	0.830
NRS [52]	learned	0.134	0.952	5.264	0.832
monodepth2 (original)	known	0.115	0.903	4.864	0.877
with our cam. model	known	0.115	0.915	4.848	0.878
with our cam. model	learned	0.113	0.885	4.829	0.878
with our cam. model $\dagger$	learned	0.109	0.826	4.702	0.884

N_g) steps. In addition, we selected 12 simulator images $\{I_{s^\alpha}\}_{\alpha \in \{f,b\}}$ and the GT depth $\{\bar{D}_{s^\alpha}\}_{\alpha \in \{f,b\}}$ for J_{depth} .

To calculate J , we set the camera height and camera model size to $h_{cam} = 0.57$ and $N_b = 32$, respectively. The weighting factors for the loss function were designed as $\lambda_1 = 0.85$, $\lambda_2 = 0.15$, $\lambda_s = 0.001$, $\lambda_f = 0.1$, and $\lambda_d = \lambda_p = 1.0$. λ_1 , λ_2 , and λ_s were exactly the same as in previous studies. We only determine λ_f by trial and error.

The robot footprint shape was defined as a circle with a diameter of 0.5 m. The point cloud of the footprint Q_{foot}^0 was set as 1400 points inside the circle. The number of steps for the robot footprints was $M_r = 5$ for J_{fel} .

The network structures of $f_{depth}()$ and $f_{pose}()$ were exactly the same as those of monodepth2 [37]. In addition, $f_{cam}()$ was designed with three convolutional layers, with the ReLU function and two fully connected layers with a sigmoid function, to estimate the camera parameters. We used the Adam optimizer with a learning rate of 0.0001 and conducted 40 epochs.

During training, we iteratively calculated J and derived the gradient to update all the models. Hence, we could simultaneously penalize J_{depth} from the simulator and the others from the real images to achieve a sim2real transfer.

For the KITTI dataset, we employed the source code of monodepth2 [37] and replaced the camera model with our proposed camera model. The other settings were exactly the same as those of monodepth2, to focus the experimentation

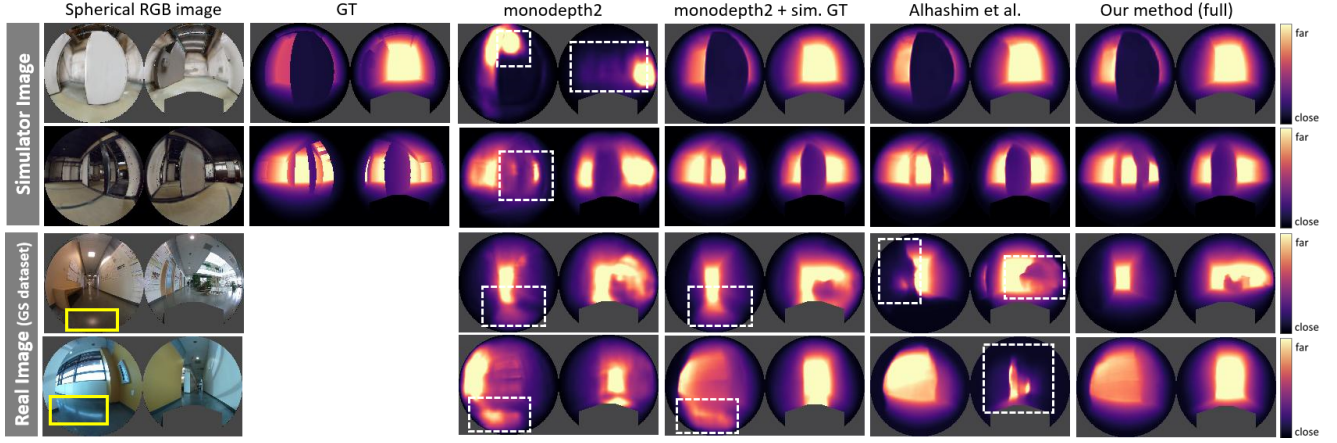


Fig. 6: Estimated depth images from 256×128 spherical camera image (GS dataset) provided by the baseline methods and our method (full). The top two rows represent the simulator image and the bottom two rows represent the real image. Yellow rectangles in the RGB image highlight the floor surface with reflected ceiling lights. White dashed rectangles on depth estimations highlight the artifacts.

on our camera model.

C. Quantitative Analysis

Table. I shows the ablation study of our method and the results of three baseline methods for comparison. We trained the following baseline methods with the same dataset.

monodepth2 [37] We applied the following half-sphere model [54] into monodepth2 [37], instead of the pinhole camera model, and trained depth and pose networks.

- back-projection: $(x_i, y_i, Z_i) \rightarrow (X_i, Y_i)$,

$$X_i = \frac{Z_i}{\sqrt{1 - x_i^2 - y_i^2}} \cdot x_i, \quad Y_i = \frac{Z_i}{\sqrt{1 - x_i^2 - y_i^2}} \cdot y_i \quad (14)$$

- Projection: $(X_i, Y_i, Z_i) \rightarrow (x_i, y_i)$

$$x_i = \frac{X_i}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}}, \quad y_i = \frac{Y_i}{\sqrt{X_i^2 + Y_i^2 + Z_i^2}} \quad (15)$$

monodepth2 with sim. GT [37], [51] We added J_{depth} to the cost function of the above baseline to train the models.

Alhashim et al. [51] We trained the depth network by minimizing J_{depth} , which is the same cost function as [51].

From Table. I, we can observe that our method significantly outperforms all baseline methods. In addition, we confirmed the advantages of our proposed components via the ablation study. The use of the GT depth from the simulator and the learning axisymmetric camera model were fairly effective. In addition, the GT depth can provide the correct scaling to the model.

In Table II, we present the quantitative results for the KITTI dataset. Similar to our method, the baseline methods (shown in Table II) learned the camera model. All methods presented in Table II used ResNet-18 for their depth network to allow for fair comparisons.

In our method with known camera parameters, we set $\{h_q\}_{q=0 \dots N_b}$, r_x , r_y , o_x , and o_y as the constant values for the GT camera's intrinsic parameters. $\{b_q\}_{q=0 \dots N_b}$ was designed with equal intervals between 0.0 and 1.0. Our method improved the accuracy of depth estimation by learning the

camera model. In addition, our method outperformed all baseline methods, including the original monodepth2.

Moreover, we trained our models by mixing the KITTI, Cityscape [55], bike [5], and GS datasets [11] to evaluate its ability to handle various cameras and evaluated on KITTI images. During training, all images were aligned into KITTI's image size by center cropping. In GS dataset, we use front-side fisheye images. Our method showed improved performance by adding datasets from various cameras with various distortions, as shown at the bottom of Table II.

D. Qualitative Analysis

Figure 6 shows the estimated depth images from simulated images and real images from the GS dataset. The depth images estimated by **monodepth2** are blurred. This is caused by the small size of the input image and the camera model's error. The GT depth from the simulator can sharpen the simulated depth of images in **monodepth2 with sim. GT**. However, there are many artifacts, particularly on the reflected floor. **Alhashim et al.** observed errors in the estimated depth from real images. **Alhashim et al.** failed sim2real transfer because the depth network was trained only from simulated images. However, our method (rightmost side) can accurately estimate depth images of both real and simulated images by reducing these artifacts. Additional examples are provided in the supplemental videos.

Finally, we present the depth images of KITTI in Fig. 1 [b]. Our method can handle the pinhole camera image and estimate an accurate depth image without camera calibration.

V. CONCLUSIONS

We proposed a novel learnable axisymmetric camera model for self-supervised monocular depth estimation. In addition, we proposed to provide supervision for the estimated depth using the GT depth from the photo-realistic simulator. By mixing real and simulator images during training, we can achieve a sim2real transfer in depth estimation. Additionally, we proposed loss functions to provide the constraints for the floor to reduce artifacts that may result from reflective floors. The effectiveness of our method was quantitatively and qualitatively validated using the GS and KITTI datasets.

REFERENCES

- [1] S. Thrun, "Probabilistic robotics," *Communications of the ACM*, vol. 45, no. 3, pp. 52–57, 2002.
- [2] J. Biswas and M. Veloso, "Depth camera based indoor mobile robot localization and navigation," in *2012 IEEE International Conference on Robotics and Automation*. IEEE, 2012, pp. 1697–1702.
- [3] T. Zhou, M. Brown, N. Snavely, and D. G. Lowe, "Unsupervised learning of depth and ego-motion from video," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 1851–1858.
- [4] V. Casser, S. Pirk, R. Mahjourian, and A. Angelova, "Depth prediction without the sensors: Leveraging structure for unsupervised learning from monocular videos," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 8001–8008.
- [5] R. Mahjourian, M. Wicke, and A. Angelova, "Unsupervised learning of depth and ego-motion from monocular video using 3d geometric constraints," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 5667–5675.
- [6] Z. Yang, P. Wang, Y. Wang, W. Xu, and R. Nevatia, "Every pixel counts: Unsupervised geometry learning with holistic 3d motion understanding," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 0–0.
- [7] N. Hirose, S. Koide, K. Kawano, and R. Kondo, "Plg-in: Pluggable geometric consistency loss with wasserstein distance in monocular depth estimation," in *2021 International Conference on Robotics and Automation (ICRA)*. IEEE, 2021.
- [8] <https://matterport.com>, (accessed August 20, 2021).
- [9] N. Hirose, A. Sadeghian, M. Vázquez, P. Goebel, and S. Savarese, "Gonet: A semi-supervised deep learning approach for traversability estimation," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 3044–3051.
- [10] N. Hirose, A. Sadeghian, F. Xia, R. Martín-Martín, and S. Savarese, "Vunet: Dynamic scene view synthesis for traversability estimation using an rgb camera," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 2062–2069, 2019.
- [11] N. Hirose, F. Xia, R. Martín-Martín, A. Sadeghian, and S. Savarese, "Deep visual mpc-policy learning for navigation," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3184–3191, 2019.
- [12] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The kitti dataset," *International Journal of Robotics Research (IJRR)*, 2013.
- [13] A. Gordon, H. Li, R. Jonschkowski, and A. Angelova, "Depth from videos in the wild: Unsupervised monocular depth learning from unknown cameras," in *Proceedings of the IEEE International Conference on Computer Vision*, 2019, pp. 8977–8986.
- [14] I. Vasiljevic, V. Guizilini, R. Ambrus, S. Pillai, W. Burgard, G. Shakhnarovich, and A. Gaidon, "Neural ray surfaces for self-supervised learning of depth and ego-motion," in *2020 International Conference on 3D Vision (3DV)*. IEEE, 2020, pp. 1–11.
- [15] R. Garg, V. K. Bg, G. Carneiro, and I. Reid, "Unsupervised cnn for single view depth estimation: Geometry to the rescue," in *European conference on computer vision*. Springer, 2016, pp. 740–756.
- [16] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised monocular depth estimation with left-right consistency," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 270–279.
- [17] X. Fei, A. Wong, and S. Soatto, "Geo-supervised visual depth prediction," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 1661–1668, 2019.
- [18] B. Ummenhofer, H. Zhou, J. Uhrig, N. Mayer, E. Ilg, A. Dosovitskiy, and T. Brox, "Demon: Depth and motion network for learning monocular stereo," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 5038–5047.
- [19] V. Guizilini, R. Ambrus, S. Pillai, and A. Gaidon, "Packnet-sfm: 3d packing for self-supervised monocular depth estimation," *arXiv preprint arXiv:1905.02693*, vol. 5, 2019.
- [20] V. Patil, W. Van Gansbeke, D. Dai, and L. Van Gool, "Don't forget the past: Recurrent depth estimation from monocular video," *arXiv preprint arXiv:2001.02613*, 2020.
- [21] Z. Yang, P. Wang, Y. Wang, W. Xu, and R. Nevatia, "Lego: Learning edge with geometry all at once by watching videos," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 225–234.
- [22] Z. Yang, P. Wang, W. Xu, L. Zhao, and R. Nevatia, "Unsupervised learning of geometry with edge-aware depth-normal consistency," *arXiv preprint arXiv:1711.03665*, 2017.
- [23] Z. Yin and J. Shi, "Geonet: Unsupervised learning of dense depth, optical flow and camera pose," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 1983–1992.
- [24] A. Johnston and G. Carneiro, "Self-supervised monocular trained depth estimation using self-attention and discrete disparity volume," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 4756–4765.
- [25] Z. Zhang, S. Lathuiliere, E. Ricci, N. Sebe, Y. Yan, and J. Yang, "Online depth learning against forgetting in monocular videos," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 4494–4503.
- [26] C. Wang, J. Miguel Buenaposada, R. Zhu, and S. Lucey, "Learning depth from monocular videos using direct methods," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2022–2030.
- [27] A. Ranjan, V. Jampani, L. Balles, K. Kim, D. Sun, J. Wulff, and M. J. Black, "Competitive collaboration: Joint unsupervised learning of depth, camera motion, optical flow and motion segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 12 240–12 249.
- [28] Y. Chen, C. Schmid, and C. Sminchisescu, "Self-supervised learning with geometric constraints in monocular video: Connecting flow, depth, and camera," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 7063–7072.
- [29] J. Hu, Y. Zhang, and T. Okatani, "Visualization of convolutional neural networks for monocular depth estimation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 3869–3878.
- [30] V. R. Kumar, S. Yogamani, M. Bach, C. Witt, S. Milz, and P. Mader, "Unrectdepthnet: Self-supervised monocular depth estimation using a generic framework for handling common camera distortion models," *arXiv preprint arXiv:2007.06676*, 2020.
- [31] V. Guizilini, J. Li, R. Ambrus, S. Pillai, and A. Gaidon, "Robust semi-supervised monocular depth estimation with reprojected distances," in *Conference on Robot Learning*, 2020, pp. 503–512.
- [32] S. Pillai, R. Ambrus, and A. Gaidon, "Superdepth: Self-supervised, super-resolved monocular depth estimation," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 9250–9256.
- [33] J. Bian, Z. Li, N. Wang, H. Zhan, C. Shen, M.-M. Cheng, and I. Reid, "Unsupervised scale-consistent depth and ego-motion learning from monocular video," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Álché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/file/6364d3f0f495b6ab9dcf8d3b5c6e0b01-Paper.pdf>
- [34] M. Poggi, F. Aleotti, F. Tosi, and S. Mattoccia, "On the uncertainty of self-supervised monocular depth estimation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [35] S. Vijayanarasimhan, S. Ricco, C. Schmid, R. Sukthankar, and K. Fragkiadaki, "Sfm-net: Learning of structure and motion from video," *arXiv preprint arXiv:1704.07804*, 2017.
- [36] M. Jaderberg, K. Simonyan, A. Zisserman, et al., "Spatial transformer networks," in *Advances in neural information processing systems*, 2015, pp. 2017–2025.
- [37] C. Godard, O. Mac Aodha, M. Firman, and G. J. Brostow, "Digging into self-supervised monocular depth estimation," in *Proceedings of the IEEE International Conference on Computer Vision*, 2019, pp. 3828–3838.
- [38] Y. Zou, Z. Luo, and J.-B. Huang, "Df-net: Unsupervised joint learning of depth and flow using cross-task consistency," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 36–53.
- [39] X. Luo, H. Jia-Bin, R. Szeliski, K. Matzen, and J. Kopf, "Consistent video depth estimation," *arXiv preprint arXiv:2004.15021*, 2020.
- [40] N. Hirose, S. Taguchi, K. Kawano, and S. Koide, "Variational monocular depth estimation for reliability prediction," *arXiv preprint arXiv:2011.11912*, 2020.
- [41] G. Yang, T. Hao Mingli Ding, N. Sebe, and E. Ricci, "Transformer-based attention networks for continuous pixel-wise prediction," *arXiv preprint arXiv:2103.12091*, 2021.
- [42] J. H. Lee, M.-K. Han, D. W. Ko, and I. H. Suh, "From big to small: Multi-scale local planar guidance for monocular depth estimation," *arXiv preprint arXiv:1907.10326*, 2019.
- [43] V. R. Kumar, S. Milz, C. Witt, M. Simon, K. Amende, J. Petzold, S. Yogamani, and T. Pech, "Monocular fisheye camera depth estimation using sparse lidar supervision," in *2018 21st International*

- Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2018, pp. 2853–2858.
- [44] C. Won, J. Ryu, and J. Lim, “Sweepnet: Wide-baseline omnidirectional depth estimation,” in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 6073–6079.
 - [45] R. Komatsu, H. Fujii, Y. Tamura, A. Yamashita, and H. Asama, “360 depth estimation from multiple fisheye images with origami crown representation of icosahedron,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 10 092–10 099.
 - [46] Z. Cui, L. Heng, Y. C. Yeo, A. Geiger, M. Pollefeys, and T. Sattler, “Real-time dense mapping for self-driving vehicles using fisheye cameras,” in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 6087–6093.
 - [47] V. R. Kumar, S. A. Hiremath, M. Bach, S. Milz, C. Witt, C. Pinard, S. Yogamani, and P. Mäder, “Fisheyedistancenet: Self-supervised scale-aware distance estimation using monocular fisheye camera for autonomous driving,” in *2020 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2020, pp. 574–581.
 - [48] V. R. Kumar, M. Klingner, S. Yogamani, S. Milz, T. Fingscheidt, and P. Mader, “Syndistnet: Self-supervised monocular fisheye camera distance estimation synergized with semantic segmentation for autonomous driving,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2021, pp. 61–71.
 - [49] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.
 - [50] F. Xia, W. B. Shen, C. Li, P. Kasimbeg, M. E. Tchappi, A. Toshev, R. Martín-Martín, and S. Savarese, “Interactive gibson benchmark: A benchmark for interactive navigation in cluttered environments,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 713–720, 2020.
 - [51] I. Alhashim and P. Wonka, “High quality monocular depth estimation via transfer learning,” *arXiv preprint arXiv:1812.11941*, 2018.
 - [52] I. Vasiljevic, V. Guizilini, R. Ambrus, S. Pillai, W. Burgard, G. Shakhnarovich, and A. Gaidon, “Neural ray surfaces for self-supervised learning of depth and ego-motion,” in *2020 International Conference on 3D Vision (3DV)*, 2020, pp. 1–11.
 - [53] D. Eigen, C. Puhrsch, and R. Fergus, “Depth map prediction from a single image using a multi-scale deep network,” in *Advances in neural information processing systems*, 2014, pp. 2366–2374.
 - [54] J. Courbon, Y. Mezouar, L. Eckert, and P. Martinet, “A generic fisheye camera model for robotic applications,” in *2007 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2007, pp. 1683–1688.
 - [55] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, “The cityscapes dataset for semantic urban scene understanding,” in *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.